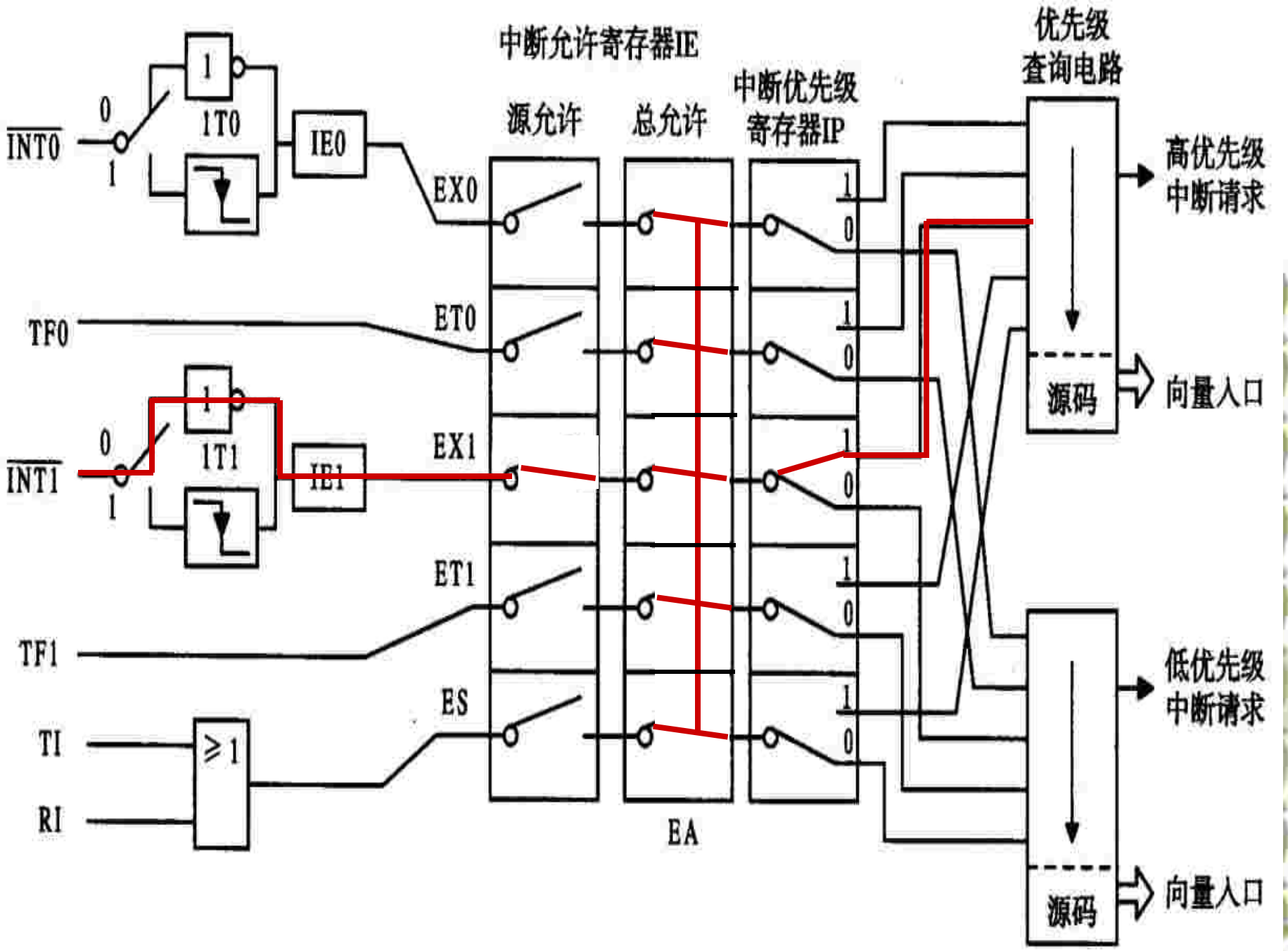
第6章 MCS-51微型计算机系统原理及应用

6.4 MCS-51中断系统与应用

§6.4.1 MCS-51中断系统

MCS-51中典型的8051单片机有5个中断源，分别为2个从INT0、INT1(P3.2、P3.3)上输入的外部中断源0和外部中断源1，2个片内定时器/计数器T/C0、T/C1的溢出中断源和1个片内串行口发送或接收中断源。


中断允许寄存器IE
优先级
查询电路
源允许
总允许
中断优先级
寄存器IP
INT0
0
1
1
1T0
IE0
EX0
TF0
ET0
INT1
1T1
IE1
EX1
ET1
TF1
ES
TI
RI
≥1
EA
高优先级
中断请求
向量入口
源码
低优先级
中断请求
向量入口
源码

1、中断源和中断标志

1）8051中断源

（1）外部中断类

外部中断是由外部原因引起的，即外部中断0（INT0）和外部中断1（INT1）：

$\overline{\text{INT0}}$——外部中断0请求信号，由P3.2引脚输入。由IT0（TCON. 0）决定中断请求信号是低电平有效还是下降沿有效。一旦输入信号有效，即向CPU申请中断，并且使IE0＝1。

$\overline{\text{INT1}}$——外部中断1请求信号，由P3.3引脚输入。由IT1（TCON.2）决定中断请求信号是低电平有效还是下降沿有效。一旦输入信号有效，即向CPU申请中断，并且使IE1＝1。

（2）定时中断：当定时器／计数器中的计数结构发生计数溢出时，即表明定时时间到或计数值已满，这时就以计数溢出信号作为中断请求，去置位一个溢出标志位，中断源可以由单片机芯片外部引入。

TF0——溢出中断请求。当定时器T0产生溢出时，其中断请求标志TF0＝1，请求中断处理。

TF1——定时器T1溢出中断请求。当定时器T1产生溢出时，其中断请求标志TF1＝1，请求中断处理。

（3）串行口中断：为串行数据的传送需要而设置的。

RI或TI——串行中断请求。当接收或发送完一串行帧数据时，使内部串行口中断请求标志RI或TI＝1，并请求中断。

2）中断标志

（1）定时器/计数器控制寄存器TCON（88H）

单元地址 88H

TCON	TF1	TR1	TF0	TR0	IE_1	IT_1	IE_0	IT_0
位地址	8FH	8EH	8DH	8CH	8BH	8AH	89H	88H

- TF1：T_1中断标志位
  - 硬件复位：0
  - T_1溢出中断：1
- TR1：T_1启停位
  - 停T_1计数：0
  - 启T_1计数：1
- TF0：T_0中断标志位
  - 硬件复位：0
  - T_0溢出中断：1
- TR0：T_0启停位
  - 停T_0计数：0
  - 启T_0计数：1
- IT_0：触发标志位
  - 0：$\overline{INT_0}$ 电平触发（软件复位）
  - 1：$\overline{INT_0}$ 负边沿触发
- IE_0：中断标志位
  - 0：硬件复位
  - 1：$\overline{INT_0}$ 上有中断
- IT_1：触发标志位
  - 0：$\overline{INT_1}$ 电平触发（软件复位）
  - 1：$\overline{INT_1}$ 负边沿触发
- IE_1：中断标志位
  - 0：硬件复位
  - 1：$\overline{INT_1}$ 上有中断

（2）串行口控制寄存器SCON（98H）

						TI	RI
						99H	98H

TI—串行口发送中断标志位。串行口发送完一组数据时，向CPU发出中断申请，同时也使TI由硬件自动置位。CPU响应此中断后，须由软件复位。

RI—串行口接收中断标志位。串行口接收到一组数据时，向CPU发出中断申请，同时也使RI由硬件自动置位。CPU响应此中断后，须由软件复位。

MCS-51系统复位后，TCON和SCON中各位被复位成“0”状态。

2、MCS-51对中断请求的控制

1）对中断允许的控制

中断允许总控位EA，配合各中断源的中断允许控制位共同实现对中断请求的控制。这些中断允许控制位集成在中断允许寄存器IE（A8H）中。

EA			ES	ET1	EX1	ET0	EX0
AFH			ACH	ABH	AAH	A9H	A8H

EA—CPU中断允许总控位,（位地址为AFH）。

当EA=1时，CPU开放中断，每个中断源是允许还是禁止分别由各自的允许位确定。当EA=0，CPU关闭所有中断请求，称关中断。

ES—串行口中断允许控制位，位地址为ACH。当ES=1，允许串行口中断，当ES=0，禁止串行口中断。

ET1—定时器/计数器T/C1的溢出中断允许控制位，位地址为ACH。当ET1=1，允许T/C1中断，当ET1=0，禁止T/C1中断。

EX1—外部中断1的中断允许控制位，位地址为AAH。当EX1=1，允许外部中断1中断，当EX1=0，禁止外部中断1中断。

ET0—定时器/计数器T/C0的溢出中断允许控制位，位地址为A9H。当ET0=1，允许T/C0中断，当ET0=0，禁止T/C0中断。

EX0—外部中断0的中断允许控制位，位地址为A8H。当EX0=1，允许外部中断0中断，当EX0=0，禁止外部中断0中断。

中断允许寄存器IE的单元地址是A8H，各控制位也可位寻址，故既可以用字节传送指令也可以用位操作指令来对各个中断请求加以控制。

例：开放T1的溢出中断

```
SETB  EA
SETB  ET1
```

```
MOV IE，#88H；
```

2）中断优先级的控制

MCS-51具有两个中断优先级，可由软件设置每个中断源为高优先级中断或低优先级中断。

IP（B8H）：中断优先级寄存器，锁存各中断优先级的控制位，用户可由软件设定。

			PS	PT1	PX1	PT0	PX0
			BCH	BBH	BAH	B9H	B8H

PS：串行口中断优先级控制位。当PS=1时，串行口中断被定义为高优先级中断，当PS=0，串行口中断被定义为低优先级中断。

PT1：T/C1中断优先级控制位。当PT1=1时，T/C1中断被定义为高优先级中断，当PT1=0，T/C1中断被定义为低优先级中断。

PX1：外部中断1中断优先级控制位。当PX1=1时，外部中断1中断被定义为高优先级中断，当PX1=0，外部中断1中断被定义为低优先级中断。

PT0：T/C0中断优先级控制位。当PT0=1时，T/C0中断被定义为高优先级中断，当PT0=0，T/C0中断被定义为低优先级中断。

PX0：外部中断0中断优先级控制位。当PX0=1时，外部中断0中断被定义为高优先级中断，当PX0=0，外部中断0中断被定义为低优先级中断。

3、MCS-51对中断的响应

单片机响应中断的条件为中断源有请求(中断允许寄存器IE=1),且CPU开中断(即EA=1)。这样,在每个机器周期内,单片机对所有中断源都进行顺序检测,并可在任1个周期的S6期间,找到所有有效的中断请求,还对其优先级进行排队。但是，必须满足下列条件：

1）中断响应的条件

(1)无同级或高级中断正在服务;

(2)现行指令执行到最后1个机器周期且已结束;

(3)若现行指令为RETI或需访问特殊功能寄存器IE或IP的指令时,执行完该指令且紧随其后的另1条指令也已执行完。

单片机在下1个机器周期的S1期间响应中断。否则,将丢弃中断查询的结果。

2）中断响应过程

① 关中断

② 保存断点

③ 转入中断入口地址

把被响应的中断源所对应的中断服务程序的入口地址（中断矢量）送入PC, 从而转入相应的中断服务程序。

中断源	入口地址
外部中断0	0003H
外部中断1	0013H
定时器/计数器T0	000BH
定时器/计数器T1	001BH
串行口中断	0023H

3）中断的响应时间

不同的情况对中断响应的时间不同：

（1）最短的响应时间，需要5个机器周期。

调用指令本身要用2个机器周期，从外部中断请求有效到开始执行中断服务程序的第一条指令，至少需要3个机器周期。

（2）如果遇到中断受阻，中断响应时间会更长一些。不同中断受阻情况，附加的等待时间为1～5个机器周期。

（3）若系统中只有一个中断源，则响应时间为3～8个机器周期。如果有两个以上中断源同时申请中断，则响应时间将更长。

4、中断请求的撤消

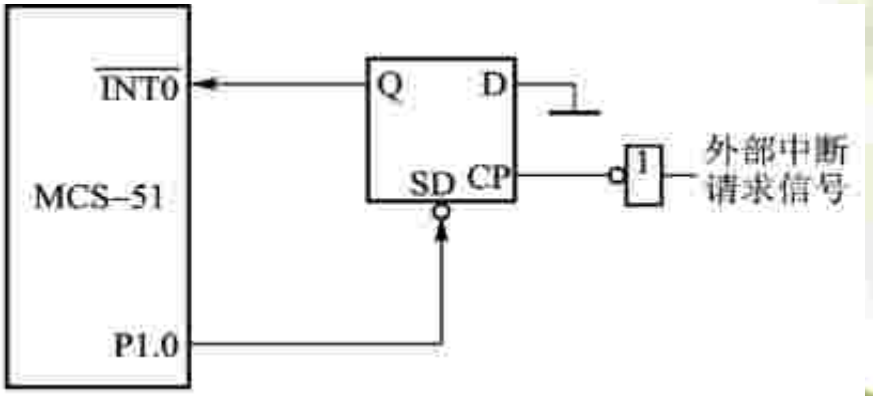
1）定时器/计数器中断请求的撤消

中断请求被响应后。硬件会自动清TF0或TF1。

2）外部中断请求的撤消

（1）跳沿方式外部中断请求的撤消是自动撤消的。

（2）电平方式外部中断请求的撤消:除了标志位清“0”之外，还需在中断响应后把中断请求信号引脚从低电平强制改变为高电平。



用P1.0接在触发器的S端作为应答线，当CPU响应中断后可使用如下指令：

```
ANL     P1，# 0FEH
ORL     P1，# 01H
CLR     IE0
```

3）串行口中断请求的撤消

响应串行口的中断后，CPU无法知道是接收中断还是发送中断，还需测试这两个中断标志位的状态，以判定是接收操作还是发送操作，然后才能清除。所以串行口中断请求的撤消只能用软件清除。

```
CLR   TI   ；清TI标志位
CLR   RI   ；清RI标志位
```

§6.4.2 MCS-51中断系统应用

1、MCS-51中断系统的初始化

中断初始化是指用户对TCON、SCON、IE等特殊功能寄存器中的各控制位进行赋值。

中断系统的初始化步骤：

① CPU开中断或关中断

② 某中断源中断请求的允许或禁止（屏蔽）

③ 设定所用中断的中断优先级

④ 若为外部中断，则应规定低电平还是负边沿的中断触发方式

例：写出INT1为低电平触发的中断系统初始化程序

位指令：

```
SETB  EA;
SETB  EX1     ;开INT1中断
SETB  PX1     ；INT1为高优先级
CLR   IT1     ；INT1低电平触发
```

字节指令：

```
MOV   IE , #84H
ORL   IP , #04H
ANL   TCON , #0FBH
```

例：写出INT0为负边沿触发的中断系统初始化程序

位指令：

```
SETB EA;
SETB EX0;开INT0中断
SETB PX0; INT0为高优先级
SETB IT0; INT0下降沿触发
```

字节指令：

```
MOV  IE, #81H
ORL  IP, #01H
ORL  TCON,#01H
```

2、中断服务程序

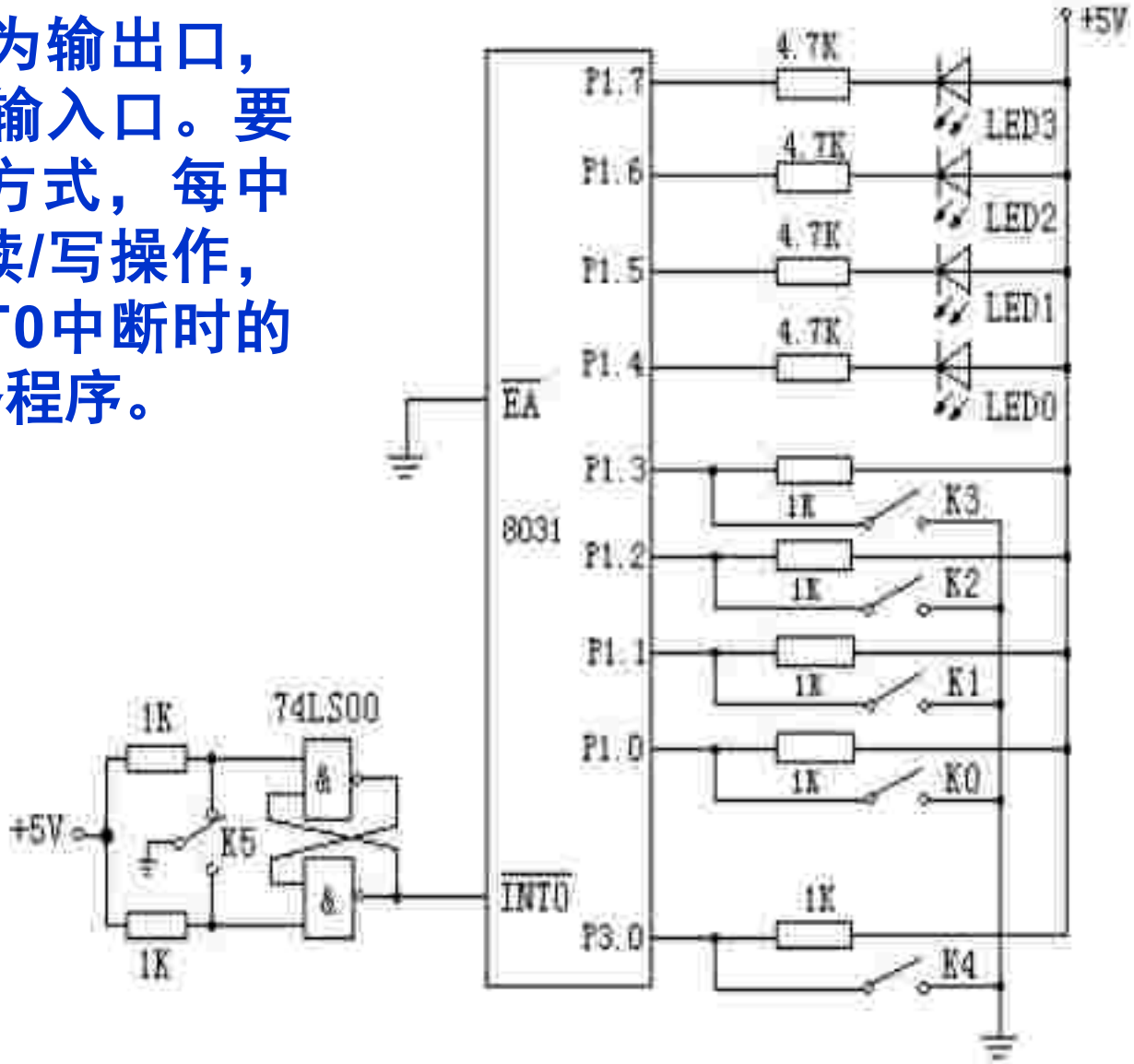
编写此程序应注意以下几点：

① 为了要跳到用户设计的中断服务程序，在相应入口地址安排一条跳转指令；

② 在中断服务程序的末尾，安排一条返回指令RETI；

③ 由于在响应中断时，CPU只自动保护断点，所以CPU的其他现场的保护和恢复也必须由用户在中断服务程序中安排。

例： P1.4—P1.7作为输出口，P1.0—P1.3作为输入口。要求采用中断触发方式，每中断一次完成一次读/写操作，写出CPU响应INT0中断时的主程序和中断服务程序。



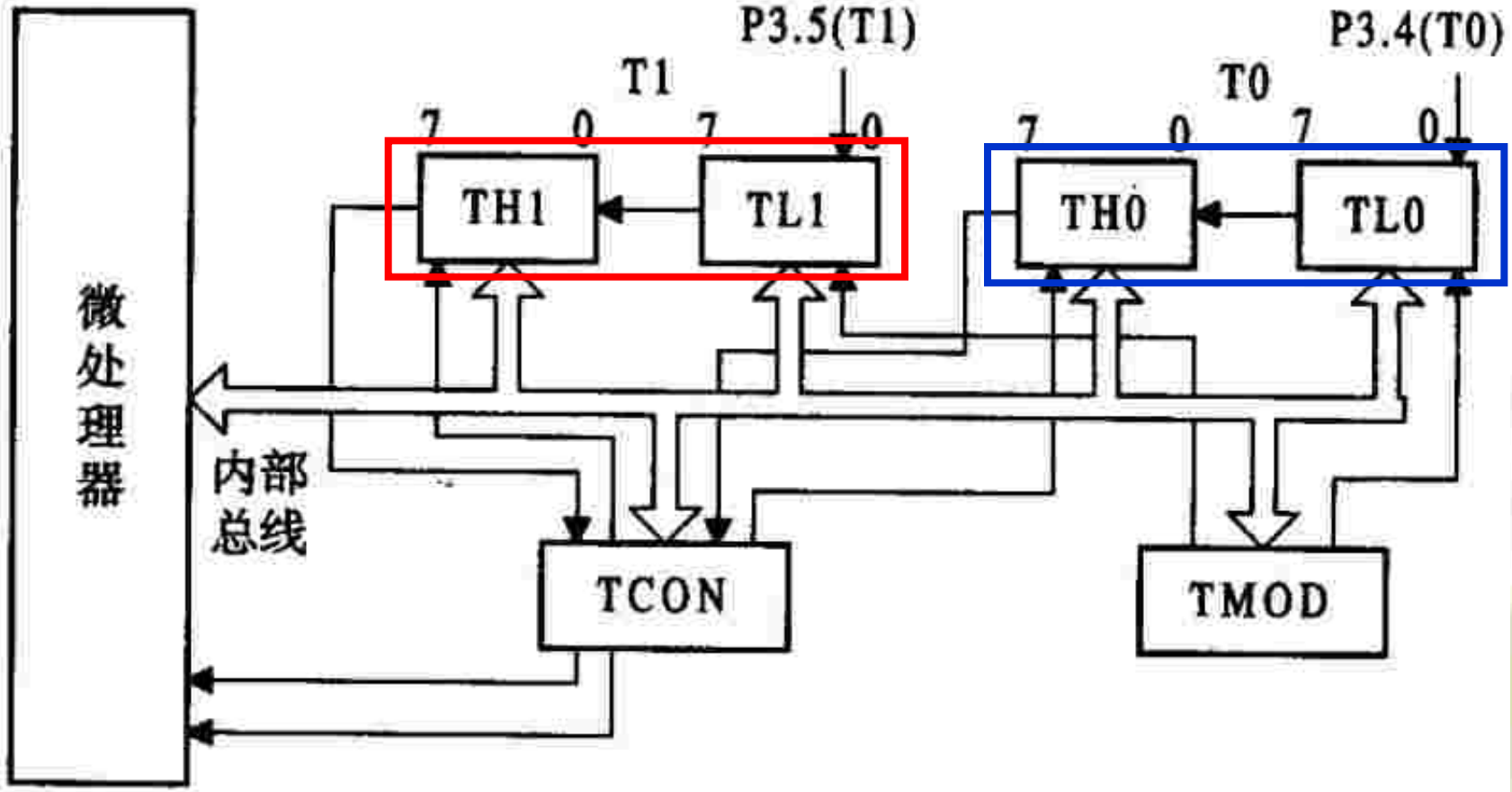
```
        ORG 0000H
        AJMP MAIN;
        ORG 0003H
        AJMP INTVS;
        …
        ORG 0100H
MAIN:   SETB EA;
        SETB EX0;
        SETB PX0;
        SETB IT0;
HERE:   AJMP HERE;
```

```
        ORG 0200H
INTVS:  MOV A, #0FFH;
        MOV P1, A;
        MOV A, P1;
        SWAP A;
        MOV P1, A;
        RETI
        END
```

6.5 MCS-51内部资源应用

§6.5.1 MCS-51内部定时计数器及其应用

MCS-51单片机内部有2个16位的可编程的定时器/计数器，即定时器/计数器0（T/C0）和定时器/计数器1（T/C1），它们都有定时和对外部事件计数的功能。T/C0是由两个8位的特殊功能寄存器TH0（8CH）和TL0（8AH）组成的，T/C1是由TH1（8DH）和TL1（8BH）组成。通过对TH1、TL1、TH0和TL0的初始化编程来控制T/C0和T/C1的计数初值。


P3.5(T1)
T1
P3.4(T0)
T0
7
0
7
0
7
0
7
0
TH1
TL1
TH0
TL0
微处理器
内部总线
TCON
TMOD

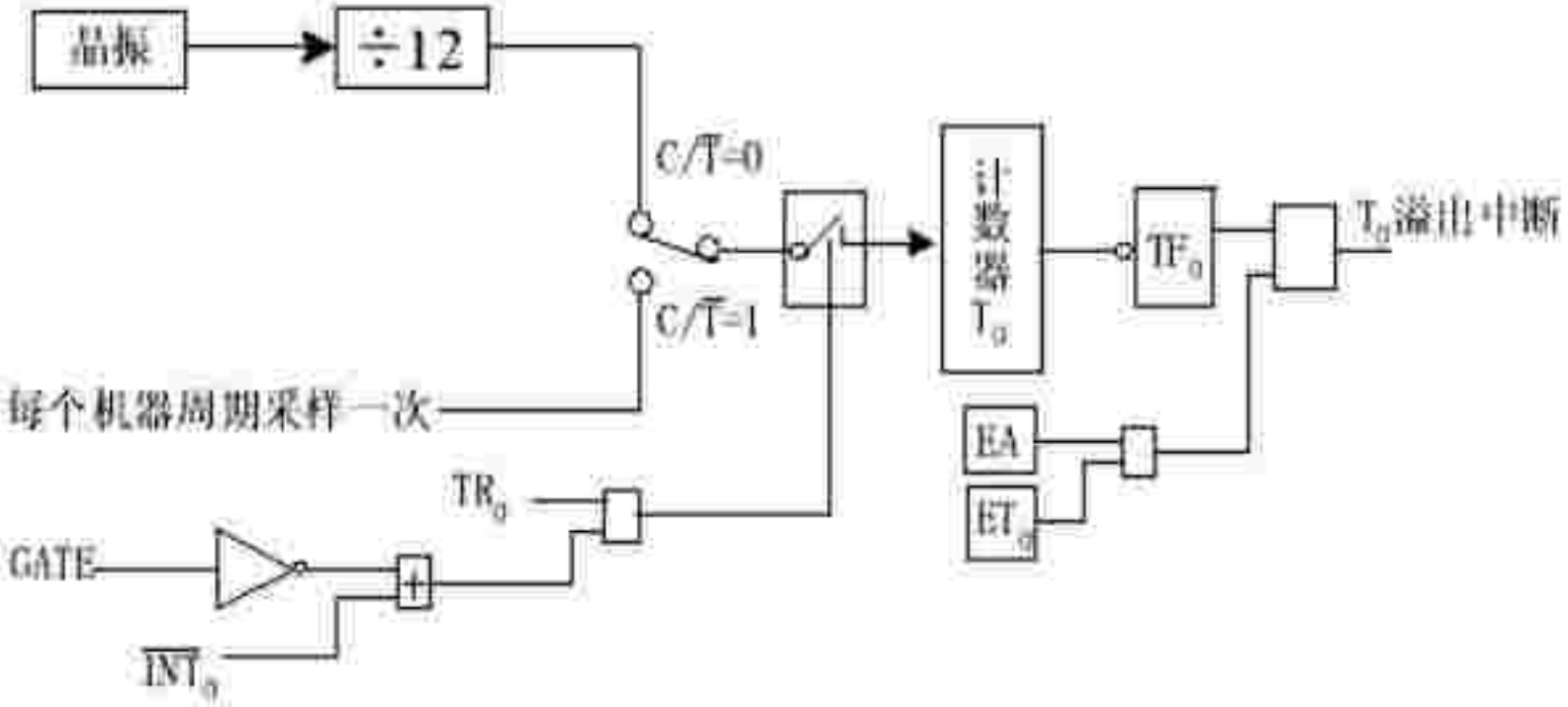
1、定时/计数器的内部结构及工作原理

加法计数器是计满溢出时才申请中断,所以在给计数器赋初值时,不能直接输入所需的计数值, 而应输入的是计数器计数的最大值与这一计数值的差值。设最大值为M,计数值为N,初值为X,则X的计算方法如下:

计数状态: X=M－N

定时状态: X=M－定时时间/T

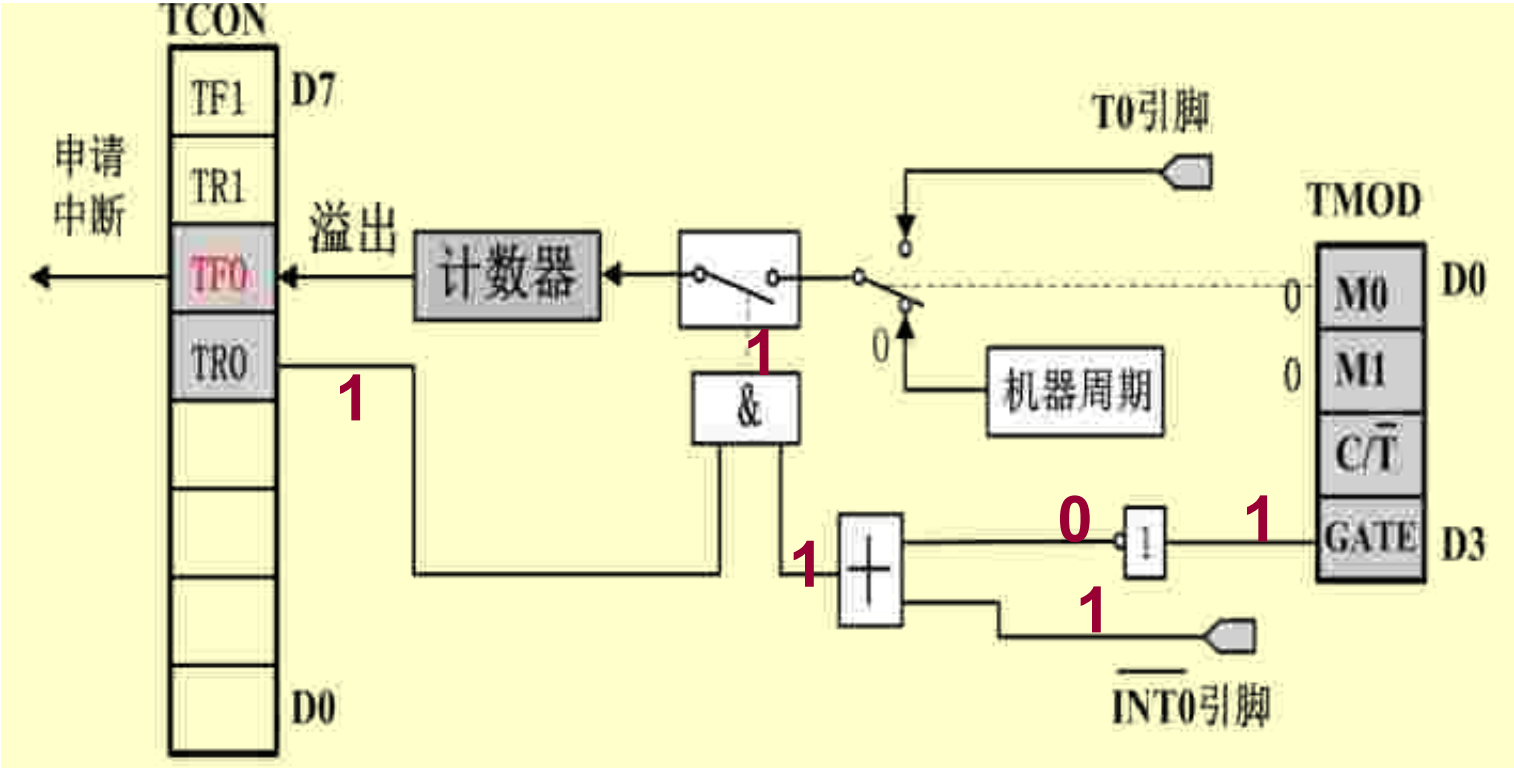
而 T=12÷晶振频率


晶振
÷12
C/T=0
C/T=1
计数器 T_0
TF_0
T_0溢出中断
每个机器周期采样一次
EA
ET_0
TR_0
GATE
$\overline{INT_0}$

2、定时/计数器工作方式及控制寄存器

1）定时器/计数器的方式寄存器TMOD

(MSB)							(LSB)
GATE	C/T	M1	M0	GATE	C/T	M1	M0
T1方式控制				T0方式控制			



M1M0工作方式控制位

M1	M0	方式	说　　明
0	0	0	13 位定时器(TH的 8 位和TL的低 5 位）
0	1	1	16 位定时器/计数器
1	0	2	自动重装入初值的 8 位计数器
1	1	3	T0 分成两个独立的 8 位计数器, T1 在方式 3 时停止工作

方式0的计数最大值：2^{13}

方式1的计数最大值：2^{16}

方式2的计数最大值：2^{8}

方式3的计数最大值：2^{8}

2）定时器/计数器控制寄存器TCON

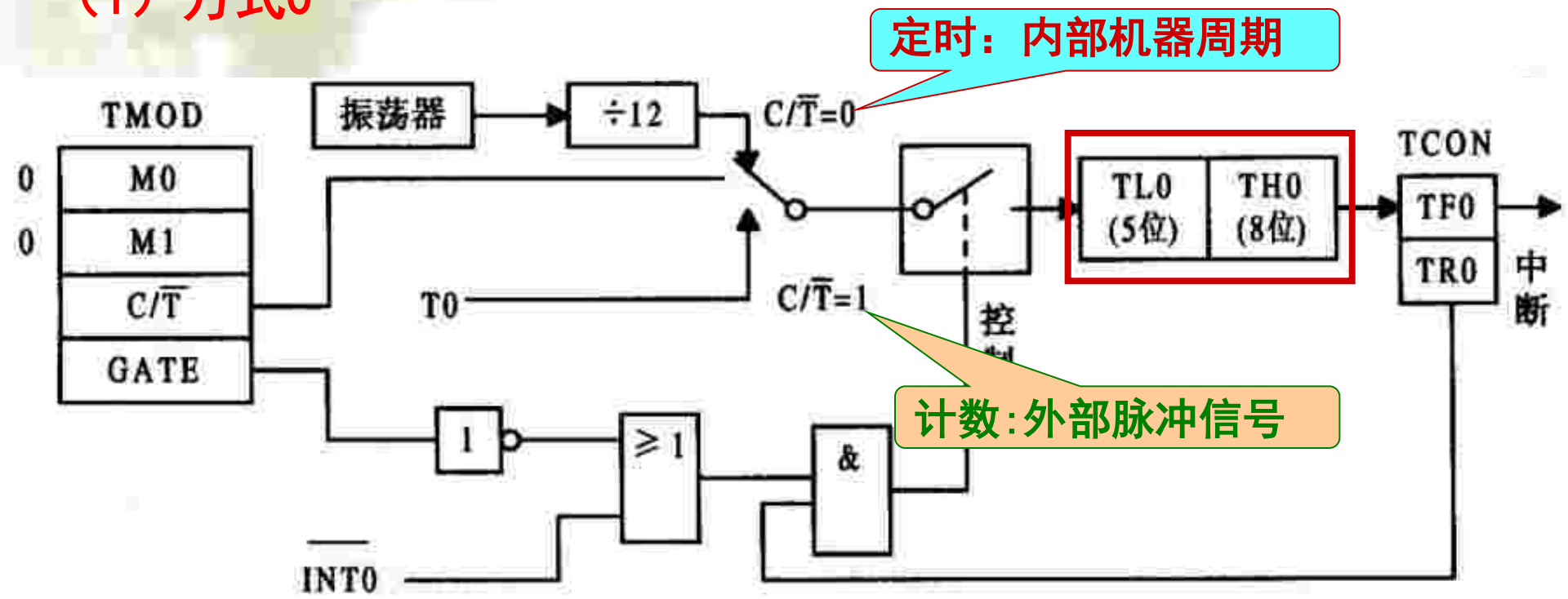
	D7	D6	D5	D4	D3	D2	D1	D0	
TCON	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0	88H

TR1、TR0分别是定时器/计数器T1、T0的运行控制位,通过**软件置1**后,定时器/计数器才开始工作,在系统复位时被清0。

TF0、TF1分别是定时器/计数器T0、T1的溢出标志位,加法计数器**计满溢出时置1**,申请中断,在中断响应后自动复0。TF产生的中断申请是否被接受,还需要由中断是否开放来决定。

3）定时/ 计数器的工作方式

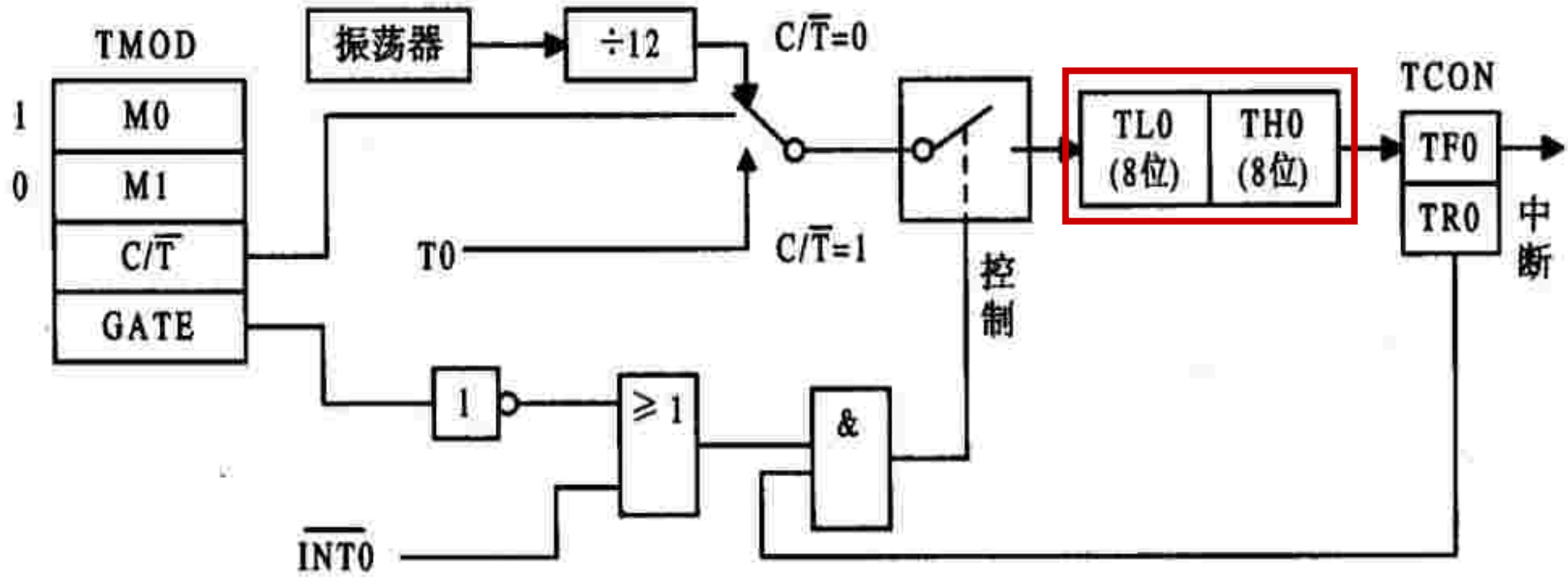
（1）方式0



方式0:高8位和低5位的13位计数器的运行方式。当TL1/TL0的低5位溢出时，向TH1(TH0)进位，而TH1(TH0)溢出时向TF1(TF0)标志进位。

（2）方式1：

是一个16位定时器/计数器，TH1（TH0）和TL1（TL0）均是8位的，构成16位计数器。



方式1（16位计数器）

（3）方式2：8位计数值自动重装

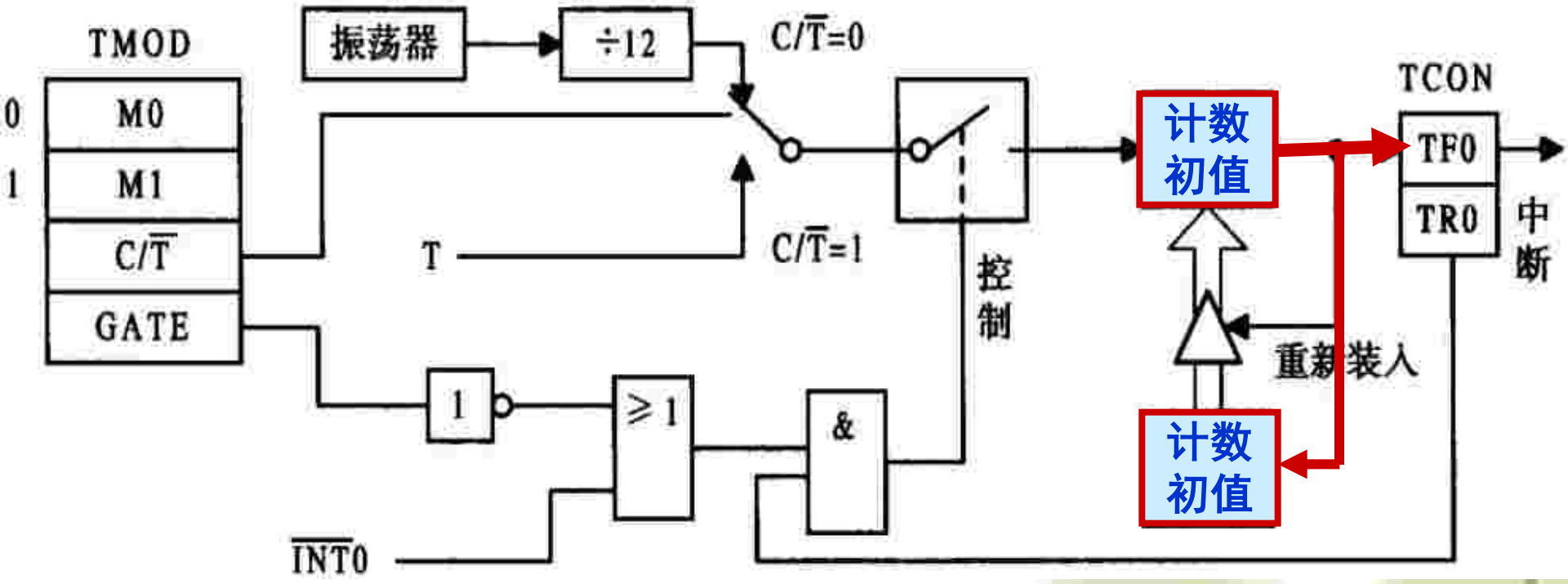
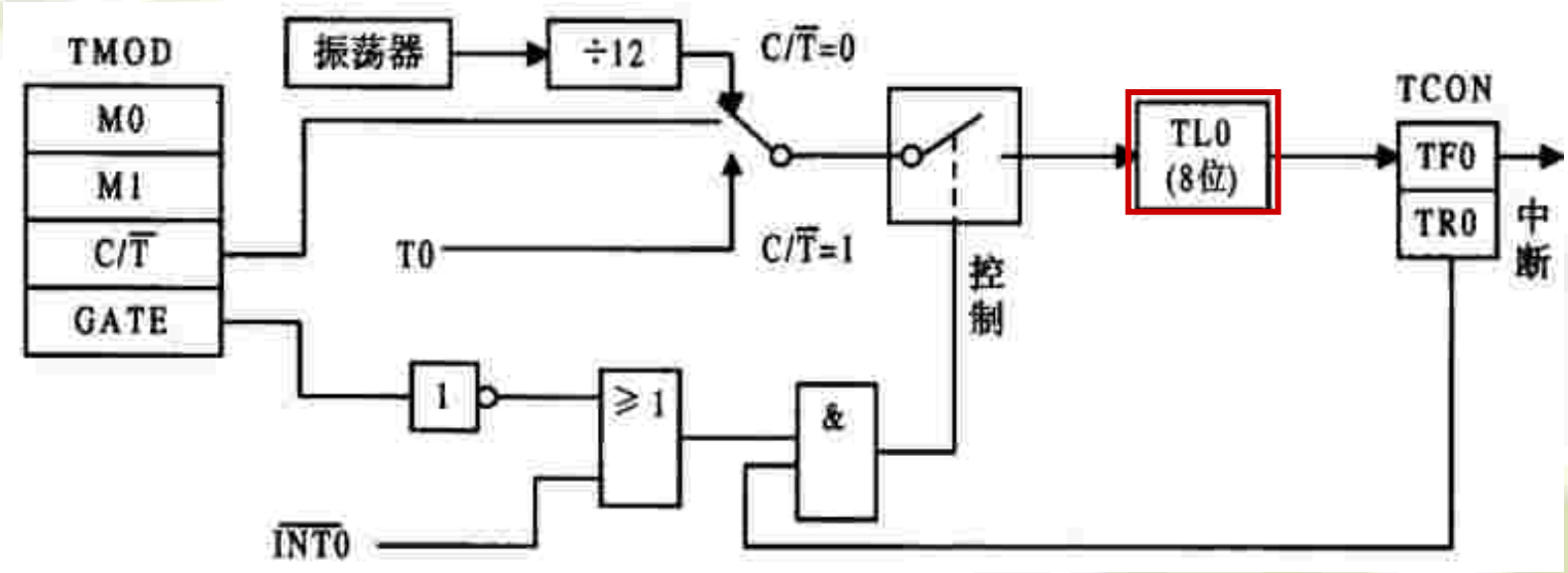
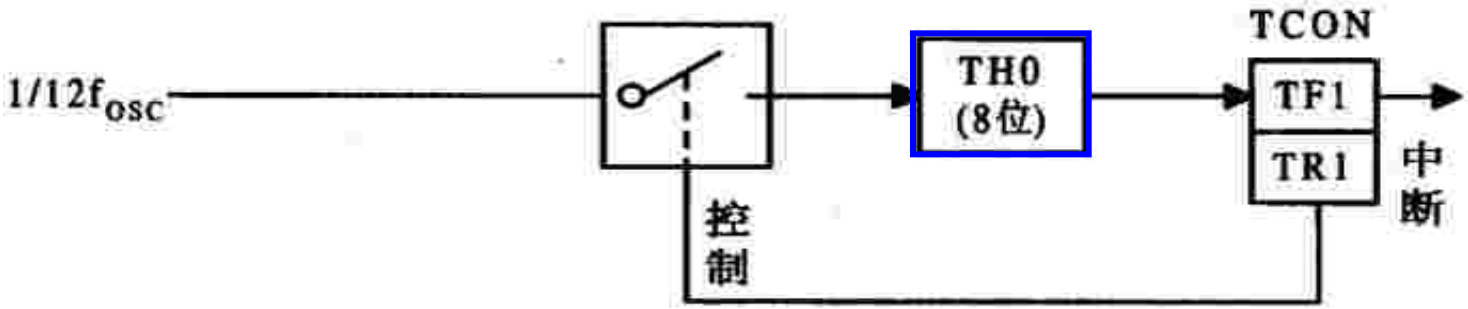


图 方式2（初始常数自动重装载）

（4）方式3

增加一个附加的8位定时器/计数器，从而具有3个定时器/计数器。





方式3只适用于定时器/计数器T0。T1不能工作在方式3，T1方式3时相当于TR1=0，停止计数（此时T1可用来作串行口波特率产生器）。

工作方式3下的T0：

T0分为两个独立的8位计数器:TL0和TH0。TL0使用T0的状态控制位C/T、GATE、TR0。而TH0被固定为一个8位定时器（不能作外部计数模式），并使用定时器T1的状态控制位TR1和TF1，同时占用定时器T1的中断请求源TF1。

4、定时/计数器的应用

（1）初始化的步骤

①确定工作方式，给TMOD送一个方式控制字；

②计算计数初值X，并写入TH0、TL0或TH1、TL1；

③根据需要置位EA使CPU开放中断，同时置位ET_X，允许T/C中断；

④给TCON送命令控制字，即置位TR_X启动T/C计数。

（2）计数器初值的计算

设需要计数器计数的个数为N，计数初值为X，则：

$$X=M-N$$

例1：利用T1的方式2对外部信号计数，要求每计满100个数，将P1.0取反。

解：

（1）选择工作方式：方式2计数模式的应用。外部信号由T1(P3.5）脚输入，每发生一次负跳变计数器加1，每输入100个脉冲，计数器产生溢出中断，在中断服务程序中将P1.0取反一次。

T1方式2的控制字：TMOD=60H。

不使用T0时，TMOD的低4位可任取，但不能使T0进入方式3，这里取全0。

（2）计算T1的初值

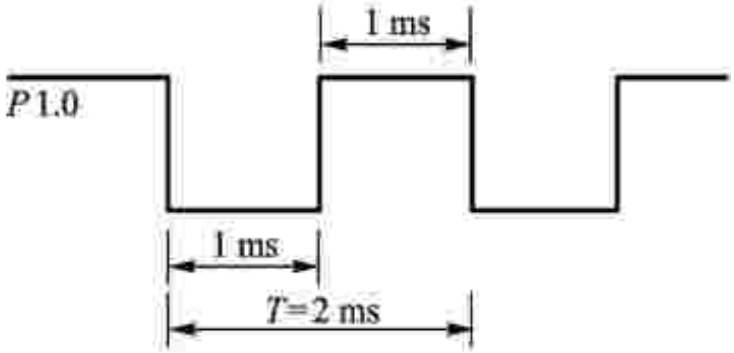
$$X=2^8-100=156=9CH$$

因此，TL1的初值为9CH，重装初值寄存器TH1=9CH

（3）程序设计

```
        ORG   0000H
        LJMP  MAIN
        ORG   001BH   ；T1中断服务程序入口
        CPL     P1.0 ；P1.0位取反
        RETI
        ORG    0100H
MAIN:   MOV    TMOD,#60H ；设T1为方式2计数
        MOV    TL0,#9CH      ；T0置初值
        MOV    TH0,#9CH
        SETB   TR1       ；启动T1
HERE:   AJMP   HERE
```

例2：利用T0方式0产生1ms定时，在P1.0引脚上输出周期为2ms的方波。设单片机晶振频率fosc=12MHZ。



方波周期为2ms，则T0定时为1ms计数溢出1次(每1ms产生一次中断)，CPU响应中断后，在中断服务程序中对P1.0取反。

(1)计算初值X

则：$2^{13}-X=1\times10^{-3}/(1\times10^{-6})$，　$X=8192-1000=7192$

即X=0001 1100 0001 1000B，所以，T0的初值为：

TH0=0E0H　TL0=18H

(2)初始化程序设计

查询方式的参考程序:

```
LOOP:   MOV   TMOD,#00H  ;  置计数器的方式控制字
        MOV   TH0,#0E0H  ;  T0置初值
        MOV   TL0,#18H
        SETB  EA
        SETB  ET0
        SETB  TR0        ;  接通T0
LOOP1:  JNB   TF0,LOOP1  ;  查询TF0标志
        CLR   TR0        ;  T0溢出，关闭T0
        CPL   P1.0       ;  P1.0的状态求反
        SJMP  LOOP
```

```
        ORG     0000H
        AJMP    MAIN
        ORG     000BH
        AJMP    TCOS
        ORG 1000H
MAIN:   MOV SP, #6FH
        MOV TMOD, #00H
        MOV TH0, #0E0H
        MOV TL0, #18H
        SETB EA
        SETB ET0
        SETB TR0
HERE:   SJMP HERE
        ……
```

中断方式的参考程序:

```
        ORG 1200H
TCOS:   CPL P1.0
        MOV TH0, #0E0H
        MOV TL0, #18H
        RETI
        END
```

例3：利用T0产生在P1.0引脚上输出周期为2s的方波。设单片机晶振频率fosc=12MHZ。

解：方波周期为2s，则T0定时应为1s，计数溢出。计数值超出最大值。

T0工作方式，定时50ms，循环20次，则实现1s定时。

(1)计算初值X

则：$2^{16}-X=50ms/(1\times10^{-6})s$，X=65536-50000=15536

TH0=3CH　TL0=B0H

(2)初始化程序设计

```
        ORG     0000H
        AJMP    MAIN
        ORG     000BH
        AJMP    BRT0
        ORG 1000H
MAIN:   MOV SP, #6FH
        MOV TMOD, #01H
        MOV TH0, #3CH
        MOV TL0, #0B0H
        SETB EA
        SETB ET0
        SETB TR0
        MOV  R0, #20
HERE:   SJMP HERE
        ............
```

```
        ORG 1200H
BRT0:   DJNZ  R0, NEXT
        CPL P1.0
        MOV  R0, #20
NEXT:   MOV TH0, #3CH
        MOV TL0, #0B0H
        RETI
        END
```

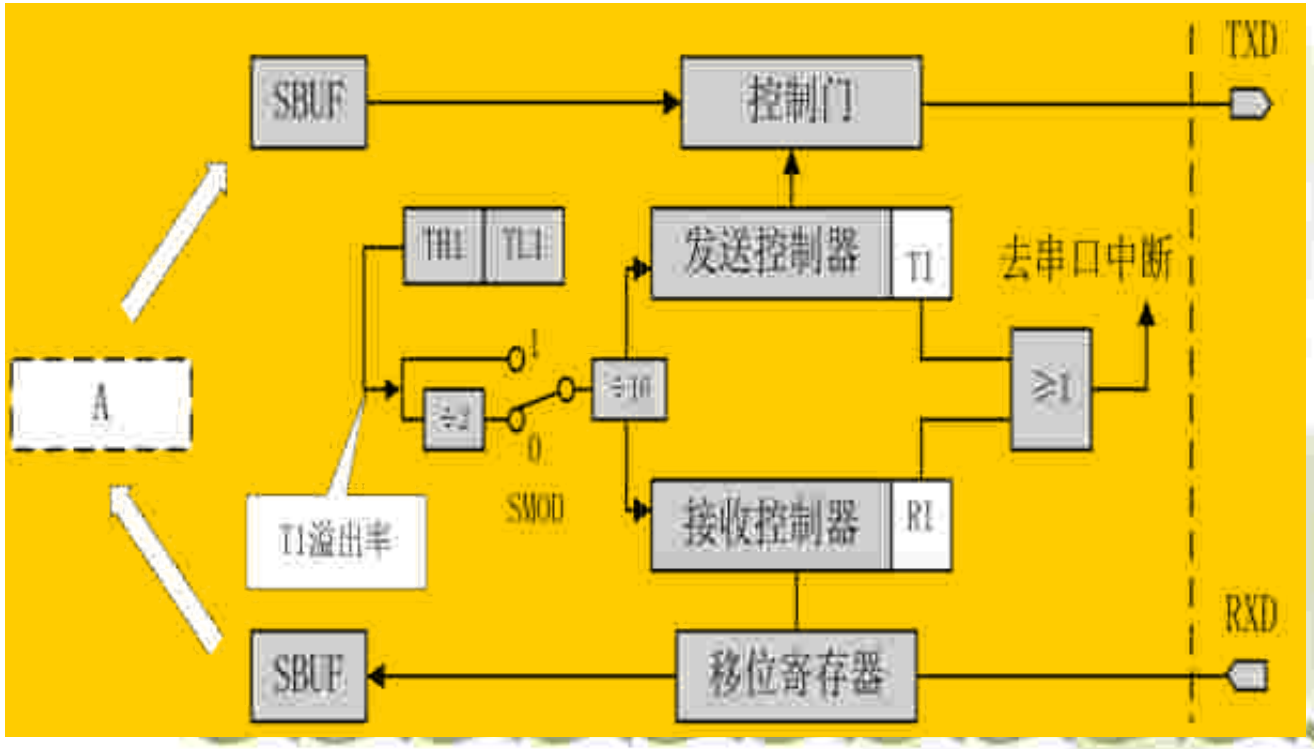
§6.5.2 MCS-51内部串行口

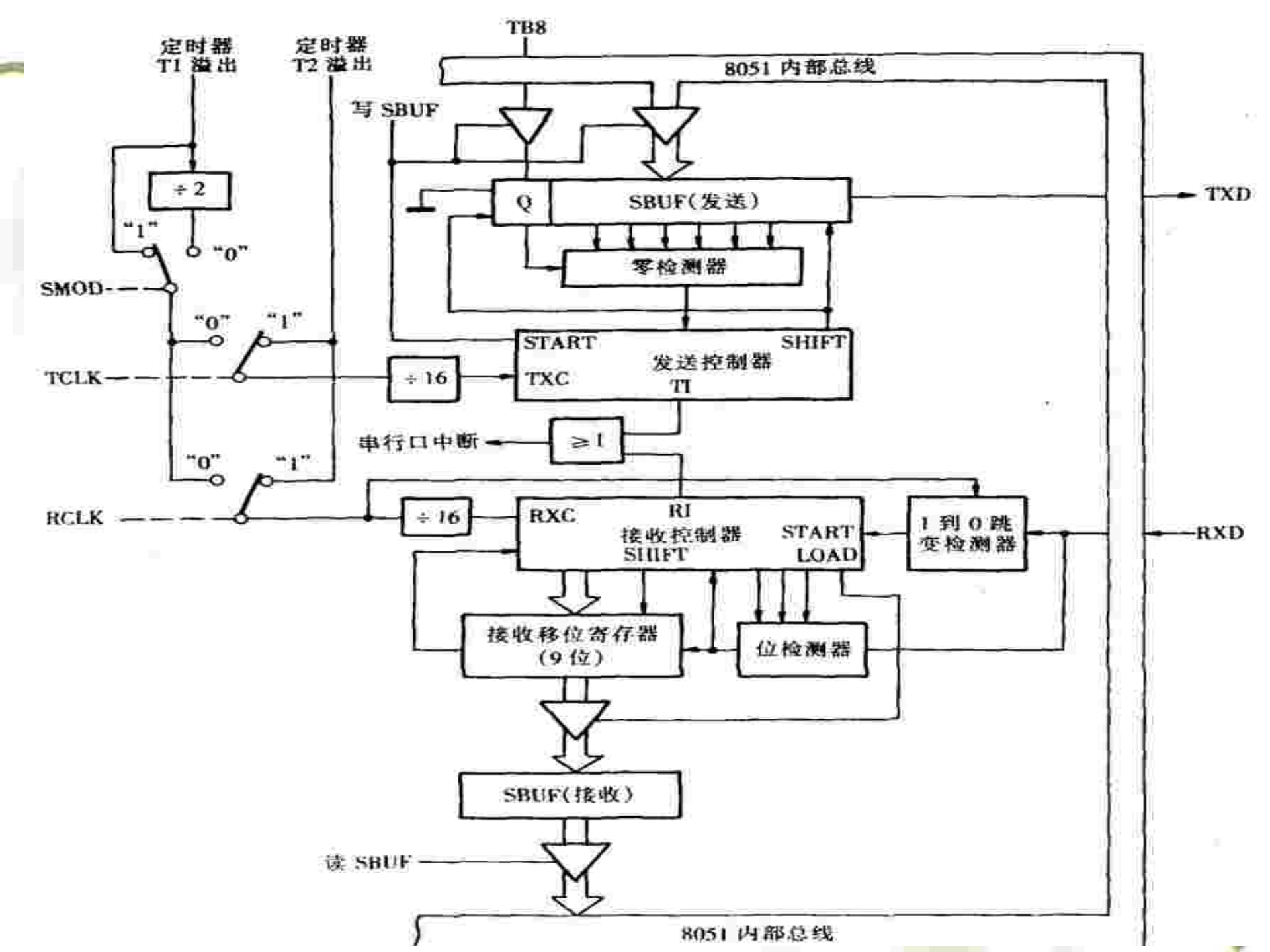
MCS-51单片机内部有1个全双工串行口，可同时发送和接收数据。它有4种工作方式，可供不同场合使用。波特率由软件设置，通过片内的定时/计数器产生。

接收、发送均可工作在查询方式或中断方式，使用十分灵活。MCS-51的串行口除了用于数据通信外，还可以非常方便地构成一个或多个并行输入/输出口，或作串/并转换，用来驱动键盘与显示器。

1、MCS-51串行口的结构

51单片机的串行口由发送/接收缓冲器以及2个专用寄存器SCON和PCON组成。用P3.0、P3.1，为RXD和TXD，从而构成全双工的通信方式。2个独立的接收、发送缓冲器SBUF（特殊功能寄存器）共用一个字节地址(99H)，用来发送和接收。




TB8
定时器
T1 溢出
定时器
T2 溢出
8051 内部总线
写 SBUF
÷ 2
"1"
"0"
SMOD
Q
SBUF(发送)
TXD
零检测器
"0"
"1"
START
SHIFT
TCLK
÷ 16
TXC
发送控制器
TI
串行口中断
≥1
"0"
"1"
RCLK
÷ 16
RXC
RI
接收控制器
SHIFT
START
LOAD
1 到 0 跳
变检测器
RXD
接收移位寄存器
(9 位)
位检测器
SBUF(接收)
读 SBUF
8051 内部总线

1）串行数据发送电路

发送电路由发送缓冲寄存器SBUF、零检测器和发送控制器等电路组成，用于串行口的发送。

2）串行数据接收电路

接收电路由接收缓冲寄存器SBUF、接收移位寄存器和接收控制器等组成，用于串行数据的接收。

3）串行口数据缓冲器SBUF

SBUF是2个在物理上独立的接收、发送缓冲器,可同时发送、接收数据。2个缓冲器只用1个字节地址99H,可通过指令对SBUF的读写来区别是对接收缓冲器的操作还是对发送缓冲器的操作。

CPU写SBUF，就是修改发送缓冲器；读SBUF,就是读接收缓冲器。串行口对外也有两条独立的收发信号线RXD(P3.0)和TXD(P3.1),因此可以同时发送、接收数据，实现全双工传送。

4）串行通信时钟

在进行异步通信时，数据的发送和接收分别在各自的时钟（TCLK和RCLK）控制下进行的，但都必须与字符位数的波特率保持一致。MCS-51串行口的发送和接收时钟可由两种方式产生，一种是由主机频率fosc经分频后产生，另一种方式是由内部定时器T1或T2的溢出率经16分频后提供。

5）发送和接收控制电路

串行发送：由指令MOV SBUF，A启动，CPU把数据(字符)写入串行口的发送缓冲器SBUF(发)中，然后在移位脉冲的控制下，低位在前，高位在后，从TXD端(方式0除外)一位位地向外发送。

串行口接收：当REN置“1”后，允许接收。接收端RXD一位位地接收数据，直到收到一个完整的字符数据后，使接收中RI置“1”，向CPU申请中断。CPU响应中断，用指令MOV A，SBUF把接收缓冲器SBUF的内容读入累加器A中。

2、串行口的工作方式

MCS-51的串行口有四种工作方式，由SCON中的SM1和SM0来决定。

1）方式0：同步移位方式

工作方式0为同步移位寄存器方式，其波特率是固定为振荡频率fosc的1/12。在这种工作方式下，发送和接收串行数据都通过RXD（P3.0）进行，从TXD（P3.1）输出移位脉冲，控制外部的移位寄存器移位。

2）方式1：双机异步通信方式

在方式1，串行口被设置为波特率可变的8位异步通信接口。

方式1传送的波特率，取决于定时器1的溢出率和特殊功能寄存器PCON中SMOD的值，计算方法如下：

方式1波特率= $(2^{SMOD}/32)$ ×定时器T1的溢出率

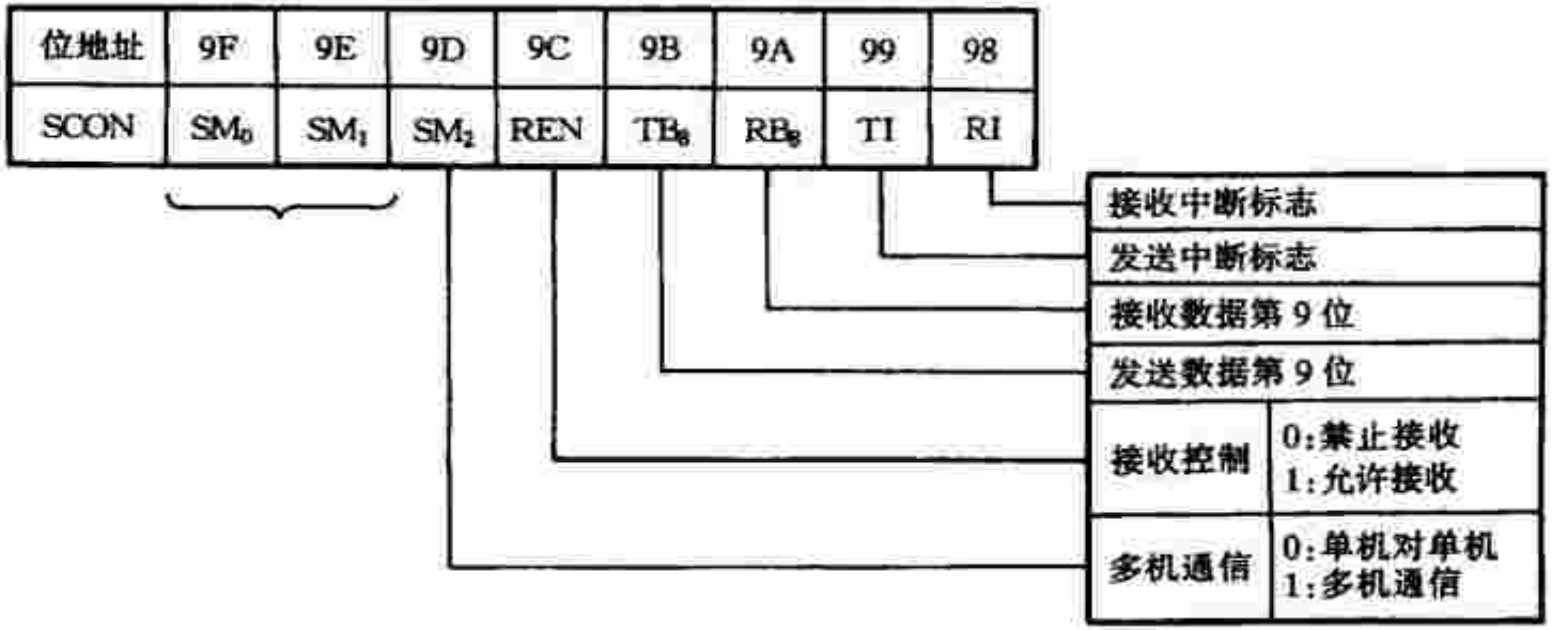
$$波特率 = \frac{2^{SMOD}}{32}\frac{f_{osc}}{12}\left(\frac{1}{2^K - N}\right)$$

当串行口置为方式1，且REN=1，RI=0时，串行口处于接收(输入状态）。

3、串行口工作方式设置

1）串行口控制寄存器SCON

SCON寄存器用来控制串行口的工作方式和状态,它可以是位寻址。在复位时所有位被清0，字地址为98H。SCON的格式为:



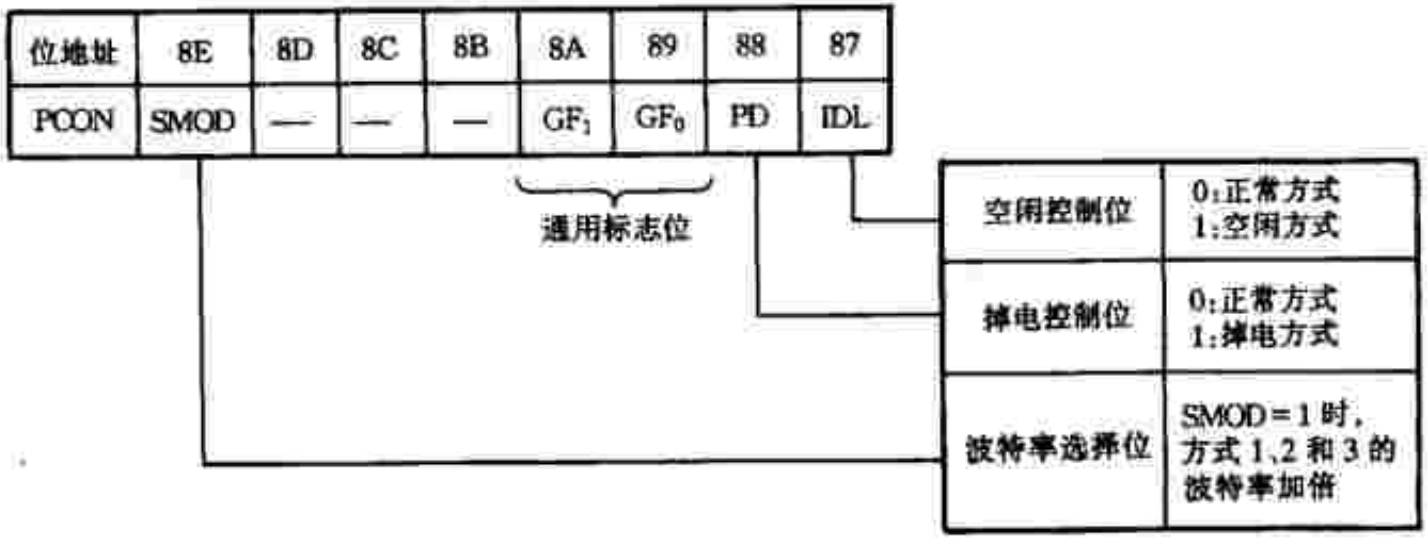
SM0、SM1: 定义串行口的操作模式, 对应4种模式。其中f_{osc}是振荡器频率, UART为通用异步接收和发送器的英文缩写。

SM0	SM1	工作方式	方式简单描述	波 特 率
0	0	0	移位寄存器 I/O	主振频率/12
0	1	1	8 位 UART	可变
1	0	2	9 位 UART	主振频率/32 或主振频率/64
1	1	3	9 位 UART	可变

SM2：多机通信控制位。主要用于工作方式2和方式3。

2） 特殊功能寄存器PCON

PCON主要是为电源控制设置的专用寄存器，字节地址为87H，不能按位寻址。



特殊功能寄存器PCON中，只SMOD与串行口的工作有关，该位是串行口波特率系数的控制位：SMOD=1 时，波特率加倍，否则不加倍。

4、串行口的工作方式及应用

MCS-51的串行口有四种工作方式，由SCON中的SM1和SM0来决定。

SM0	SM1	模　式	功　能	波　特　率
0	0	0	同步移位寄存器	$f_{osc}/12$
0	1	1	8位UART	可变(T1溢出率)
1	0	0	9位UART	$f_{osc}/64$ 或$f_{osc}/32$
1	1	1	9位UART	可变(T1溢出率)

1）方式0

工作方式0为同步移位寄存器方式，其波特率是固定为振荡频率fosc的1/12。在这种工作方式下，发送和接收串行数据都通过RXD（P3.0）进行，从TXD（P3.1）输出移位脉冲，控制外部的移位寄存器移位。

三、串行口的波特率

串行口每秒钟发送或接收的数据位数称为波特率。 假设发送一位数据所需时间为T, 则波特率为 1/T。

(1) 方式0的波特率：单片机晶振频率的 1/12, 即每个机器周期接收或发送一位数据。

(2) 方式2的波特率：PCON的最高位SMOD的写入值有关:

方式2波特率=$(2^{SMOD}/64)\times fosc$

即：SMOD=0, 波特率为$(1/64)f_{osc}$;

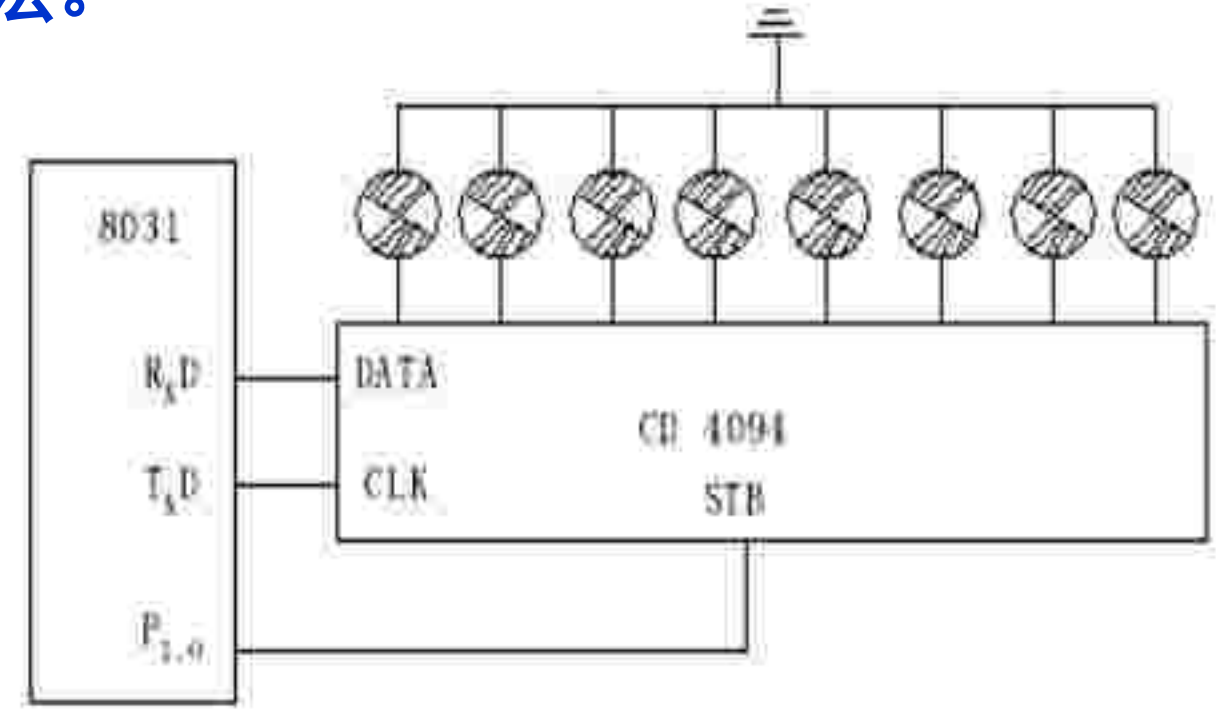
SMOD=1, 波特率为$(1/32)f_{osc}$。

(3) 方式1和方式3的波特率，除了与SMOD位有关之外，还与定时器T1的溢出率有关。定时器T1作为波特率发生器，常选用定时方式2(8位重装载初值方式)，并且禁止T1中断。此时TH1从初值计数到产生溢出，它每秒钟溢出的次数称为溢出率。 于是

$$\text{方式 1或3的波特率} = \text{T1的溢出率} \times \frac{2^{SMOD}}{32}$$

$$= \frac{2^{SMOD}}{32} \times \frac{f_{OSC}}{12 \times (256 - TH1)}$$

例：根据如下图的线路连接，请编出发光二极管自左至右以一定速度显示的程序。设发光二极管为共阴极接法。



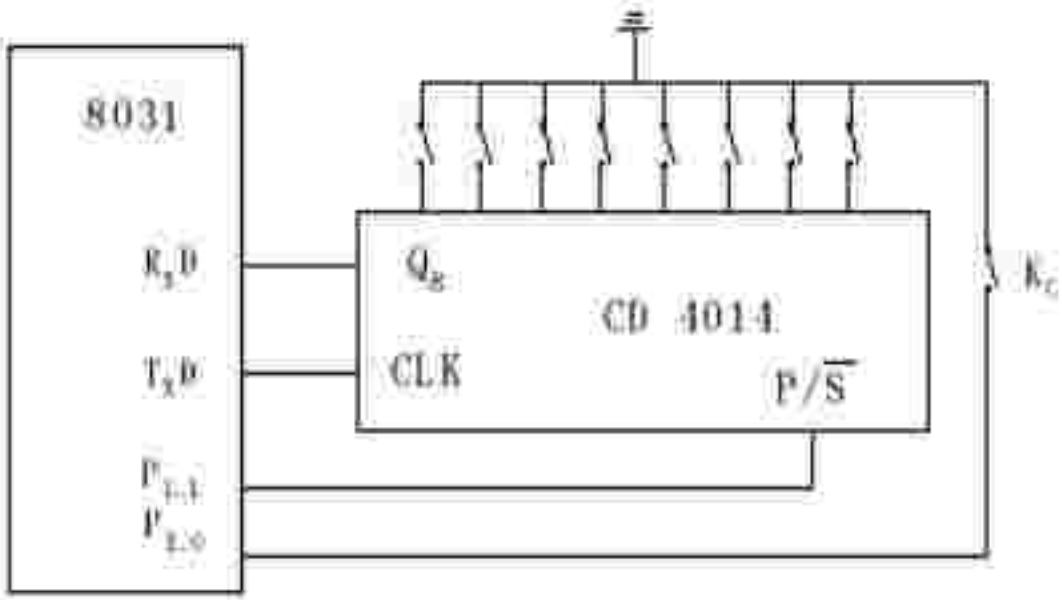
分析：CD4094是一种8位串行输入（DATA端）并行输出的同步移位寄存器。CLK为同步脉冲输入端。STB为控制端：若STB=0，则8位并行数据输出关闭，但允许串行数据从DATA输入；若STB=1，则DATA输入端关闭，但允许8位数据并行输出。设串行口采用中断方式发送，发光二极管的显示时间依靠延时程序DELAY实现。整个程序由主程序和中断服务程序两部分组成。

```
        ORG     2000H
        MOV     SCON, #00H      ;串行口初始化为方式0。
        CLR     P1.0            ;禁止CD4094并行输出。
        MOV     A, #80H         ;起始显示码送A。
        MOV     SBUF, A         ;8031串行输出。
LOOP:   SJMP    LOOP            ;等待串行口输出完。
```

中断服务程序代码：

```
        ORG     0023
        LJMP    SBV                 ;转SBV。
        ORG     0100H
SBV:    SETB    P1.0                ;点亮发光二极管。
        LCALL   DELAY               ;点亮一段时间。
        CLR     TI                  ;清发送中断标志。
        RR      A                   ;准备点亮下一位。
        CLR     P1.0                ;灭显示。
        MOV     SBUF, A             ;串行口输出。
        RETI                        ;中断返回。
DELAY:  MOV   R7, #05H
WAIT1:  MOV   R6, #0FFH
WAIT2:  DJNZ   R6, WAIT2
        DJNZ   R7, WAIT1
        RET
        END
```

例： 根据如图电路，编出MCS-51串行输入开关量并把它存入20H单元的程序。要求控制开关KC断开（KC=1）时，8031处于等待状态，KC合上（KC=0）时8031开始输入和进行模拟。



分析：CD4014 是并行输入串行输出的同步移位寄存器。其中，Q8 为串行输出端，CLK为同步移位脉冲输入端，P/S为控制端。若P/S=0 ，则CD4014可以串行输出（并行输入端关闭）；若P/S，则 CD4014 可以并行输入数据（串行输出端关闭）。程序采用对P1.0查询，查询到 KC闭合时再通过对 P1.1的控制而完成开关量的输入。

```
        ORG     2000H
START:  JB      P1.0, $         ;若KC断开，则等待。
        SETB    P1.1            ;令CD4014并行输入开关量。
        CLR     P1.1            ;CD4014开始串行输出。
        MOV     SCON, #10H      ;令串行口方式为0，启动接收。
        JNB     RI,   $         ;等待接收。
        CLR     RI              ;若接收已完，则清RI。
        MOV     A,    SBUF      ;开关量送累加器A。
        MOV     20H, A          ;存入内存。
        SJMP    START           ;准备下次开关量输入。
        END
```

2）方式1

在方式1，串行口被设置为波特率可变的8位异步通信接口。

方式1传送的波特率，取决于定时器1的溢出率和特殊功能寄存器PCON中SMOD的值，计算方法如下：

方式1波特率= $(2^{SMOD}/32)$ ×定时器T1的溢出率

当串行口置为方式1，且REN=1，RI=0时，串行口处于接收(输入状态）。

例：用中断法编出8051单片机串行口方式1下的发送程序：fosc=6MHz，定时器T1做波特率发生器，波特率为2400bps，选SMOD=1；发送字符块在内存RAM中的TBLOCK单元，字符块长度为20。采用奇校验，校验位在数据第8位。

解：SM0、SM1=01时为方式1，在SM2=0，允许发送数据，其余各位均取0。则

SCON=0100000B=40H

通过查表确定X=F4H

对于PCON：由于SMOD=1，所以PCON=80H

(同系统复位以后的状态,可不赋值)。

对于TMOD：由于只用T1，且为定时方式2，所以

TMOD=00100000B=20H

主程序：

```
MOV    TMOD, #20H      ;定时器T1设为方式2
MOV    TL1，#0F4H      ;装入定时器初值
MOV    TH1，#0F4H      ;8位重装值
SETB   TR1             ;启动定时器T1
MOV    SCON，#40H      ;串行口设为方式1
MOV    PCON, #80H      ;SMOD=1
MOV    R0，  #TBLOCK   ;发送缓冲区首址
MOV    R2，#20         ;数据长度
MOV    A，R2
MOV    SBUF, A         ;先输出字符长度
SETB   EA              ;开中断
SETB   ET1             ;允许T1中断中断
SETB   ES              ;允许串行口中断
SJMP   $               ;等待中断
```

中断服务程序：

```
        ORG    0023H        ；串行中断入口
        LJMP   TXSV         ；转至中断服务程序
TXSV:   CLR    ES           ；清串行口中断
        CLR    TI           ；清TI
        MOV    A, @R0       ；发送字符
                            ；形成奇校验C

        MOV    SBUF, A      ；启动发送
        DJNZ   R2, NEXT     ；数据未发送完成
        CLR    ES
        SJMP   $
NEXT:   INC    R0           ；修改地址指针
        RETI                ；中断返回
```

3） 方式2和方式3

串行口工作为方式2时，被定义为9位异步通信接口。

方式2波特率=$(2^{SMOD}/64)$fosc

方式3为波特率可变的9位异步通信方式，除了波特率有所区别之外，其余都与方式2相同。

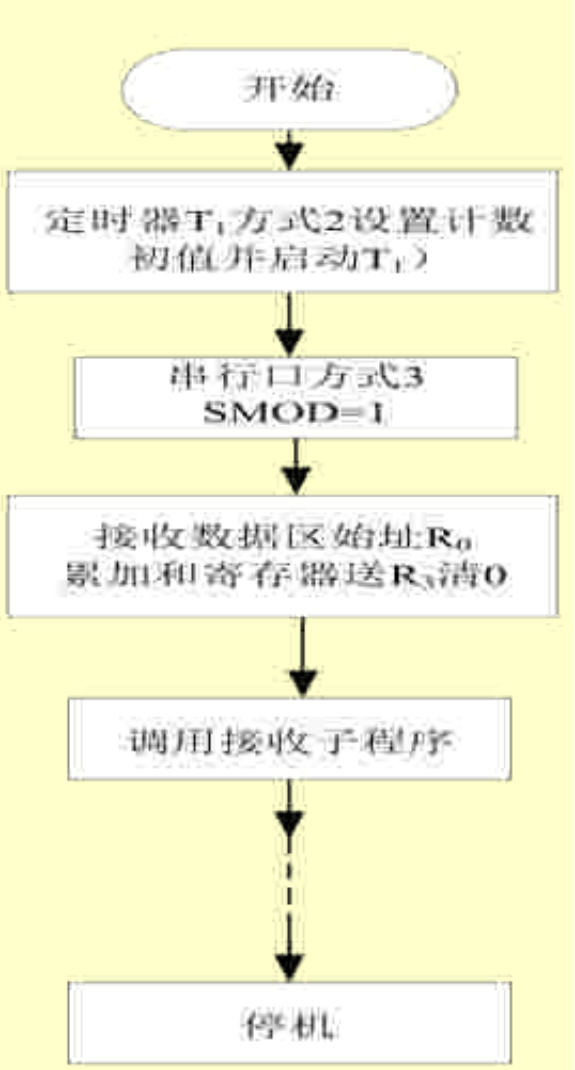
方式3的波特率=$(2^{SMOD}/32)$×(定时器T1的溢出率)

常用波特率和定时器T1初值

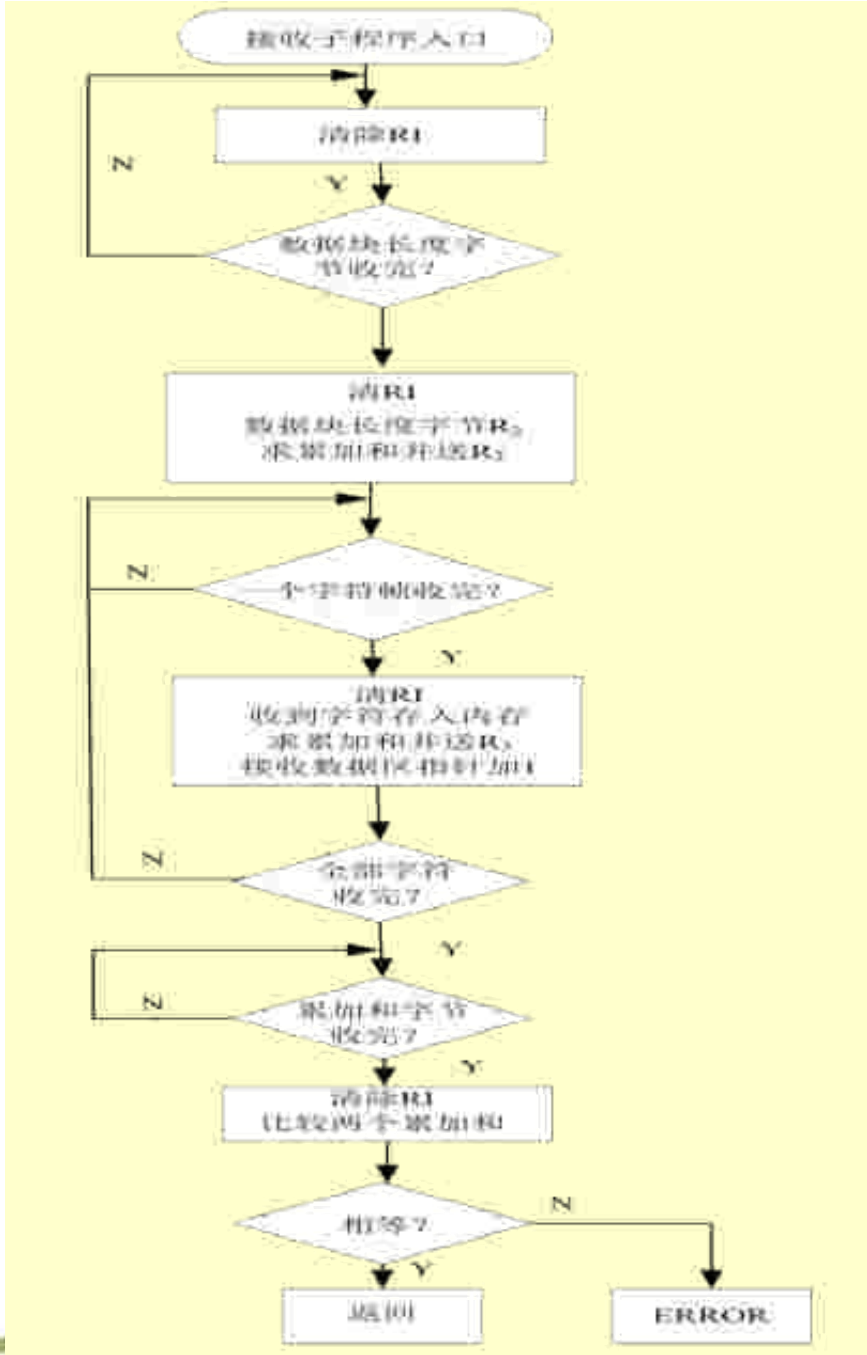
波特率		f_{osc}	SMOD	定时器 T1		
				C/T	方式	初值
方式 0	0.5 M	6 MHz	×	×	×	×
方式 2	187.5 K	6 MHz	1	×	×	×
方式 1、3	19.2 K	6 MHz	1	0	2	FEH
	9.6 K	6 MHz	1	0	2	FDH
	4.8 K	6 MHz	0	0	2	FDH
	2.4 K	6 MHz	0	0	2	FAH
	1.2 K	6 MHz	0	0	2	F4H
	0.6 K	6 MHz	0	0	2	E8H
	110	6 MHz	0	0	2	72H
	55	6 MHz	0	0	1	FEEBH

例：用查询法编出串行口在方式3下的接收程序。设单片机主频为6MHz，波特率为2400bps，接收数据区始址为RBLOCK（内部RAM），接收数据块长度字节由始发端发送来。要求采用累加和校验，若传输有错，转出错处理程序。

分析：累加和是指累加所有需要发送或接收的数据（字符）字节后得到的低字节和（大于255部分舍去）。本程序有主程序、接收子程序和出错处理程序组成。



（a)主程序流程图



（b)接收子程序流程图

主程序代码如下：

```
            ORG     1000H
RBLOCK  DATA    30H
START:  MOV     TMOD,   #20H    ;T1工作于方式2
        MOV     TH1,    #0F4H   ;设置时间常数初值
        MOV     TL1,    #0F4H
        SETB    TR1             ;启动T1
        MOV     SCON,   #0D0H   ;串行口工作于方式3接收
        MOV     PCON,   #80H    ;使SMOD=1
        MOV     R0,     RBLOCK  ;接收数据区始址送R0
        MOV     R3,     #00H    ;累加和寄存器清零
        ACALL   RXSUB           ;转接收子程序
        …
        SJMP    $               ;停机
```

接收子程序：

```
RXSUB: CLR   RI              ;清R
RXLEN: JNB   RI,    $        ;等待接收数据块长度字节
       CLR   RI              ;接收完后清RI
       MOV   A,     SBUF     ;数据块长度字节送A
       MOV   R2,    A        ;存入R2
       ADD   A,     R3       ;开始求累加和
       MOV   R3,    A        ;累加和存入R3
RxD:   JNB   RI,    $        ;等待接收字符
       CLR   RI              ;接收完后清RI
       MOV   A,     SBUF
       MOV   @R0,   A        ;接收字符存入内存
       ADD   A,     R3       ;求累加和
       MOV   R3,    A        ;存入R3
       INC   R0              ;接收数据区指针加1
       DJNZ  R2,    RxD      ;若数据块未收完，则继续
```

```
RXSUM: JNB   RI,    $        ;等待接收累加和
       CLR   RI              ;接收完后清RI
       MOV   A,    SBUF      ;接收到的累加和送A
       XRL   A,    R3        ;比较两个累加和
       JNZ   ERROR           ;若不等，则转出错处理
       RET                   ;若相等，则返回
```